COMP4611 Project: Detect and Resolve Pipeline Hazards

In this project, you will implement hazard detection and resolution for the **Pipeline Processor with Multicycle Operations** covered in Lecture 5 (L5).

Complete four functions in `src/hazards.c`: `check_raw_hazard`, `check_waw_hazard`, and `check_wb_hazard` return instruction decode (ID) stall counts; `check_control_hazard` returns an instruction fetch (IF) stall count. The simulator, assembly parser, coordination of the functions, and `Scoreboard` updates are provided.

1. Work through two instructions without forwarding

```
ADD R1, R2, R3    # Write R2 + R3 to R1
ADD R4, R1, R5    # Read that R1 value; write R1 + R5 to R4
```

Assume these are the first two instructions, R2, R3, and R5 are ready, and forwarding is disabled. After IF and ID, each `ADD` spends one cycle in each of execution (EX), memory access (MEM), and writeback (WB). **Without forwarding, sources are read in ID; WB writes before ID reads in the same cycle.**

The framework calls `check_control_hazard` once during each instruction's **first IF cycle**. During each instruction's **first ID cycle**, it calls `check_raw_hazard`, `check_waw_hazard`, then `check_wb_hazard`, once each. The cells show their returned stall counts; **0 means no additional stall**.

Instruction	Cycle 1	2	3	4	5	6	7	8
`ADD R1, R2, R3`	**IF** `check_control_hazard`: 0	**ID** `check_raw_hazard`: 0 `check_waw_hazard`: 0 `check_wb_hazard`: 0	**EX**	**MEM**	**WB** Write R1			
`ADD R4, R1, R5`		**IF** `check_control_hazard`: 0	**ID: stall** `check_raw_hazard`: 2 `check_waw_hazard`: 0 `check_wb_hazard`: 0	**ID: stall**	**ID: complete**	**EX**	**MEM**	**WB** Write R4

To detect dependencies, the framework records each register's latest writer opcode and scheduled WB cycle in `Scoreboard`. For the first `ADD`, it records `ADD` / cycle 5 for R1. In cycle 3, `check_raw_hazard` reads that record and returns 2. The framework passes candidate ID cycle 5 to `check_waw_hazard` and then `check_wb_hazard`; both return 0 because no previous instruction writes R4 or reaches WB in cycle 8. The framework then updates `Scoreboard`: `writers[4]` becomes `{ADD, 8}`, and `wb_reserved[8]` becomes `true`.

`check_control_hazard` returns 0 because no branch is pending. The first `ADD` also has no destination or WB conflict.

2. Understand the inputs and register records

The three functions called in ID share this interface, differing only in function name:

```
int check_raw_hazard(int cycle, Op inst, int src1, int src2, int dst,
                     bool forwarding, const Scoreboard *state);
```

Parameter	Meaning
`cycle`	Candidate cycle for completing ID. `check_raw_hazard` receives the current cycle; the next two functions receive it plus stalls already returned by preceding functions.
`inst`	Instruction opcode, such as `ADD` or `LW`; `Op` is the opcode type
`src1`, `src2`	Source register indices
`dst`	Destination register index
`forwarding`	`true` enables all three forwarding paths (MEM→EX, WB→EX, WB→MEM); `false` disables all three
`state`	Pointer to the read-only `Scoreboard` described below

The framework supplies valid inputs and maintains `state`. Your functions use it but cannot modify it through the `const Scoreboard *` parameter.

The parser supplies the opcode and register indices. Unlike assembly's `destination, source1, source2` order, these functions receive **source 1, source 2, then destination**.

Return the minimum number of additional ID stall cycles needed to resolve the hazard checked by this function. Return 0 if no stall is needed. The return value `stall` advances the candidate ID completion cycle to `cycle + stall`.

Register indices and `Scoreboard`

Index	Meaning
`-1`	No register in this operand position; ignore it
`0`	R0 is constant zero; its reads and writes create no register dependency
`1`–`31`	Writable integer registers R1–R31
`32`–`63`	Writable floating-point registers F0–F31; **F0 is writable**

src/hazards.h provides:

```
#define NUM_REGS 64
#define NUM_CYCLES 100064

typedef struct {
    Op writer_op;     // Opcode of the latest scheduled writer.
    int write_cycle;  // Its scheduled WB cycle.
} RegisterWriter;

typedef struct {
    RegisterWriter writers[NUM_REGS];
    bool wb_reserved[NUM_CYCLES];
} Scoreboard;
```

- `state->writers[r]` stores register `r`'s latest **writer** (or **producer**): its opcode and WB cycle number. The record changes only when the framework schedules another writer for that register; it is not cleared at WB. Initially, `write_cycle` is 0, meaning the initial value is available and `writer_op` is ignored.
- `state->wb_reserved[c]` is `true` when an earlier instruction is scheduled to reach WB in cycle `c`.

3. Apply the hardware timing and forwarding rules

Execution latency and shared stages

`Op` identifier (assembly spelling where different)	EX cycles
`ADD`, `SUB`, `ADDI`	1
`LW`, `FLD` (`L.D`)	1
`SW`, `FST` (`S.D`)	1
`FADD` (`ADD.D`), `FSUB` (`SUB.D`)	4
`FMUL` (`MUL.D`)	7
`FDIV` (`DIV.D`)	24
`BEQ`, `BNE`	1

After completing ID in cycle `c`, an instruction starts EX in `c + 1`, spends its specified latency `L` there, then one cycle in MEM and one in WB. Thus WB is in `c + L + 2`. There are no further stalls after entering EX.

Arithmetic results appear at the end of the final EX cycle; load results appear at the end of MEM. Derive forwarding availability from `writer_op` and `write_cycle`; no separate availability timestamp is supplied.

All execution units are fully pipelined. The divider takes 24 EX cycles but accepts a new division every cycle. Independent divisions can enter EX consecutively; instructions needing a division's result may still require a data-dependency stall.

Instruction and data memory are separate, and memory operations execute in program order. Every instruction—including arithmetic, stores, branches, and writes to R0—passes through MEM and WB, even without a data-memory access or register write. Each stage accepts at most one instruction per cycle.

Forwarding paths and operand use

Path	Value supplied when forwarding is enabled
MEM→EX	An arithmetic producer in MEM supplies its completed EX result to a consumer beginning EX
WB→EX	A producer in WB supplies its writeback value, including loaded data, to a consumer beginning EX
WB→MEM	A producer in WB supplies its writeback value to a store's data input in MEM

Producer and consumer must occupy the corresponding stages **in the same cycle**. A load cannot use MEM→EX: its result appears at the end of MEM, too late for EX in that cycle. Regardless of forwarding, WB writes before ID reads.

Ordinary sources, including branch operands, are needed at the beginning of the first EX cycle. For `SW` and `FST`:

- `src1` is the address base, needed in EX.
- `src2` is the data, needed in MEM. It can be read in ID, forwarded to EX and saved until MEM, or supplied directly through WB→MEM.

4. Implement and combine the four functions

`check_raw_hazard`, `check_waw_hazard`, and `check_wb_hazard`

Each function checks **only the hazard it is responsible for** and returns the additional ID stall cycles needed:

Function	Minimum ID stall to return
`check_raw_hazard`	Delay ID so both source operands are available when needed, through the register file or enabled forwarding paths.
`check_waw_hazard`	Delay ID so this instruction's WB cycle is **strictly later** than the destination's recorded `write_cycle`. Return 0 if there is no writable destination.
`check_wb_hazard`	Delay ID so this instruction does not reach WB in the same cycle as an earlier instruction.

The common signature does not require every function to use every parameter: `check_raw_hazard` checks source operands and forwarding, `check_waw_hazard` checks destination write order, and `check_wb_hazard` checks `state->wb_reserved`.

How the framework combines the returned stalls

The framework uses this sequence during the first ID cycle:

Function	`cycle` passed by the framework
`check_raw_hazard`	Current cycle
`check_waw_hazard`	Current cycle + stall returned by `check_raw_hazard`
`check_wb_hazard`	Current cycle + stalls returned by `check_raw_hazard` and `check_waw_hazard`

Total ID stall is the sum of the three additional stalls. The three functions read the same `Scoreboard`; only the `cycle` argument advances between calls. After all three return, the framework updates `Scoreboard`: it sets the final WB cycle's `wb_reserved` entry to `true` and, if the destination is writable, stores the opcode and final WB cycle in `writers[dst]`.

`check_control_hazard`: return a fetch stall count

```
int check_control_hazard(int cycle, int branch_id_cycle);
```

Parameter	Meaning
`cycle`	First cycle in which the processor attempts this fetch
`branch_id_cycle`	Branch's ID completion cycle, including all ID stalls; `-1` means no branch is pending

The framework supplies valid inputs.

A branch resolves at the **end of EX**. The next instruction can be fetched in the following cycle. **We do not consider branch prediction or early branch resolution.**

Return the minimum number of IF stall cycles needed before the next instruction can be fetched, or 0 if fetching is already allowed.

An example with ready branch operands and no other delays:

Instruction	Cycle 1	2	3	4	5	6	7	8
`BEQ`	**IF** `check_control_hazard`: 0	ID	EX: resolve	MEM	WB			
Next correct instruction		**IF: stall** `check_control_hazard`: 2	IF: stall	IF	ID	EX	MEM	WB

`check_control_hazard(2, 2)` returns 2: fetch stalls in cycles 2–3 and resumes in cycle 4.

5. Build, test, submit, and understand the marks

Build and test

You need a C99 compiler, `make`, and Python 3.8 or later. Read `src/hazards.h` and **edit only** **`src/hazards.c`**. Run the following commands from the project root directory, which contains `Makefile`:

Command	Purpose
`make`	Build the provided simulator with your functions
`make score`	Report each function's public-test stall-score percentage
`make test`	Check exact stalls and unchanged input records, then run public pipeline tests if those checks pass

The starter compiles but fails tests until implemented. SCRIPTS.md covers options, diagnostics, and optional assembly tests.

At end-of-file, fetching stops and instructions already in the pipeline finish. A taken branch may also end the program by targeting an end-of-file label.

Submission and scoring

Submit **only** **`hazards.c`**, with your name, ITSC email, and acknowledgements in its opening comment.

Function	Public	Hidden	Total marks
`check_raw_hazard`	15	15	30
`check_waw_hazard`	12.5	12.5	25
`check_wb_hazard`	10	10	20
`check_control_hazard`	12.5	12.5	25
Total	**50**	**50**	**100**

Marks depend only on returned stall counts. Input-preservation and full-program checks are debugging diagnostics and do not directly deduct marks; preserving read-only inputs remains a requirement.

Each call's **optimal stall count** is the minimum needed to resolve the hazard checked by that function. All four functions use the same scoring rule:

Returned result	Call credit
Optimal stall	100%
1 / 2 extra cycles	90% / 80%
Each further extra cycle, through 9 extra	Deduct another 10 percentage points
10 or more extra cycles	0%
Too few cycles, negative/invalid return, crash, or timeout	0%

Each function's public and hidden scores are calculated separately: calls requiring a positive optimal stall contribute **80%**, and zero-stall calls contribute **20%**, with equal weight per call within each category. For example, `check_raw_hazard`'s public percentage is 80% of the positive-stall calls' average credit plus 20% of the zero-stall calls' average, applied to its 15 public marks.

Each graded call uses predefined inputs and is scored independently. `check_control_hazard` is graded on its fetch stall alone. Thus `make score` measures stall-count accuracy, not complete-program correctness.